Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #12,
CS@TUCN

Agenda

- **Graphs isomorphism (review for comparison)**
  - with metaprogramming
  - With nonmonotonic reasoning
  - Various implementations
- **Hamiltonian cycle**
  - What is it
  - Class of the problem
  - How (not to) solve it
  - How to address the issue (next lecture)
- **BFS using Queues**

Graphs isomorphism problem statement

- Let us:
  - $G1=(V1,E1)$
  - $G2=(V2,E2)$
  - G1 iso G2 iff

    i) $|V1|=|V2|$

    ii) $\exists\ \varphi : V1 \rightarrow V2$ a bijection so that

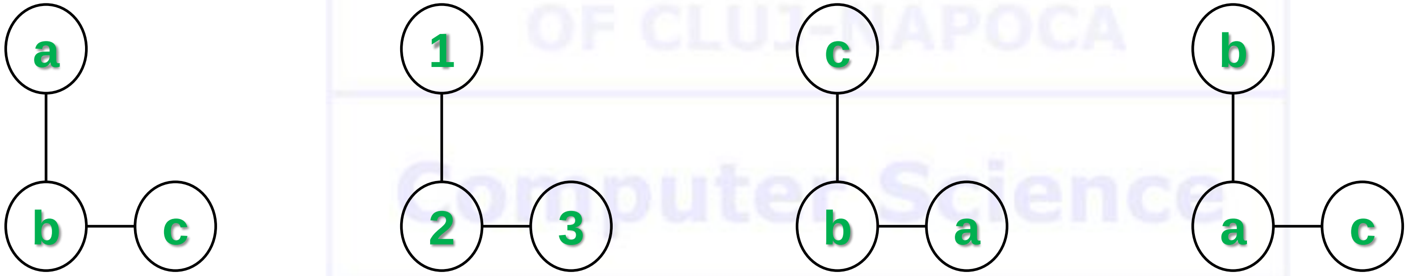
    iii) $\forall\ x, y \in V1\ [(x, y) \in E1 \Leftrightarrow (\varphi(x), \varphi(y)) \in E2)$
- In simpler words:
  - Graphs are essentially “the same” in structure, differing only by the names or arrangement of their vertices.
  - φ matches up vertices one-to-one while preserving adjacency
- Ex: Are the 2 graphs isomorphs?

```
graph1([
        n(a,[b,c,d]),
        n(b,[a,c]),
        n(c,[a,b,d]),
        n(d,[a,c])
]).
```

```
graph2([
        n(1,[2,4]),
        n(2,[1,3,4]),
        n(3,[2,4]),
        n(4,[1,2,3])
]).
```

Graphs isomorphism
Simple example

- Consider these two graphs:
  - **Graph A** has vertices labelled {a,b,c} with edges (a,b),(b,c).
  - **Graph B** has vertices labelled {1,2,3} with edges (1,2),(2,3).



Graphs isomorphism approach (v1)

```
?-graph1(G1),graph2(G2),iso_graph1(G1,G2).

iso_graph1(L1,L2):-eq_perm(L1,L2,eq_neighb).

eq_perm([H1|T1],L2,EQ):-     % is the perm predicate with an equivalence
        delete(H2,L2,T2),    % nondeterministic behavior. Implemented how?
        P=..[EQ,H1,H2],      % which is created here (metapredicate)
        call(P),             % and called here to execute
        eq_perm(T1,T2,EQ).
eq_perm([],[],_).            % succeeds iff sets have the same cardinality
% so an immediate improvement would check cardinalities beforehand

eq_neighb(n(N1,L1),n(N2,L2)):-     % 2 neighb pairs are equivalent
        eq_node(N1,N2),            % if the nodes are equivalent
        eq_perm(L1,L2,eq_node).    % and the neighb lists are equivalent

delete(H,[H|T],T).                 % what if we add a cut here?
delete(X,[H|T],[H|R]):-
        delete(X,T,R).             % what if we add the [] clause?
```

Nodes equivalence
(implements φ function)
V1 – default logic with side effects

- `p/2` a dynamic predicate, used for the nomonotonic reasoning
- implements φ function (to form `p` as akb)

```
eq_node(N1,N2):-      % if nodes N1 and N2 already form a pair in the akb
      p(N1,N2).       % the evaluation continues
eq_node(N1,_):-       % if N1 forms a pair with some OTHER node
      p(N1,_),!,      % we get an inconsistency, so should NOT allow
      fail.           % (N1,N2) form a pair, so, fail to backtrack
eq_node(_,N2):-       % symmetric on N2
      p(_,N2),!,      % not an injective function
      fail.           % How/where bijection is insured?
eq_node(N1,N2):-      % if you reach here, the is no inconsistency
      asserta(p(N1,N2)).          % hence pair N1, N2 is legitimate.
eq_node(N1,N2):-                  % if at a later moment,
      retract(p(N1,N2)),!,        % although pair N1, N2 is legitimate,
                          % the reasoning cannot conclude,
                          % so remove it from the akb, and
      fail.               % fail to backtrack and resume execution
                          % WITHOUT the pair in the akb
```

Graphs isomorphism approach (v1)

```
[q] ?- graph1(G1), graph2(G2), iso_graph1(G1,G2).

iso_graph1(L1,L2):- eq_perm(L1,L2,eq_neighb).

eq_perm([H1|T1],L2,EQ):-
        delete(H2,L2,T2),
        P=..[EQ,H1,H2],
        call(P),
        eq_perm(T1,T2,EQ).
eq_perm([],[],_).

eq_neighb(n(N1,L1),n(N2,L2)):-
        eq_node(N1,N2),
        eq_perm(L1,L2,eq_node).

eq_node(N1,N2):-p(N1,N2).
eq_node(N1,_):-p(N1,_),!,fail.
eq_node(_,N2):-p(_,N2),!,fail.
eq_node(N1,N2):-asserta(p(N1,N2)).
eq_node(N1,N2):-retract(p(N1,N2)),!,fail.

delete(H,[H|T],T).
delete(X,[H|T],[H|R]):-delete(X,T,R).
```

Nodes equivalence
(implements φ function)
V2 – with arguments (lists)

```
?- graph1(G1),graph2(G2), iso_graph2(G1,G2).
```

% instead of the akb `p` in v1, here we store the pairs in the arg list
% arg 4 = partial list; empty initially (arg to model the akb)
% arg 5 = final list; a copy of the partial list at the end of the execution
% (arg with the status of the akb at the end; the bijection)

```
iso_graph2(L1,L2):-eq_perm(L1,L2,eq_neighb,[],Lout).

eq_perm([H1|T1],L2,EQ,LI,LO):-      % same as in v1
      delete(H2,L2,T2),
      P=..[EQ,H1,H2,LI,Lint],       % just that with args
      call(P),
      eq_perm(T1,T2,EQ,Lint,LO).
eq_perm([],[],_,L,L).

eq_neighb(n(N1,L1),n(N2,L2),LI,LO):-
      eq_node(N1,N2,LI,Lint),
      eq_perm(L1,L2,eq_node,Lint,LO).
```

Nodes equivalence
(implements φ function)
V2 – with arguments (lists) – contd.

- p is a pair of nodes in the list argument
- list evolves nomonotonic (during reasoning) to allow for pairs addition/removal
- models φ function

```
eq_node(N1,N2,LI,LI):-
      member(p(N1,N2),LI), !.
eq_node(N1,N2,LI,[p(N1,N2)|LI]):-
      not(member(p(N1,_),LI)),
      not(member(p(_,N2),LI)).
```

- Those 2 clauses do the same as those 5 in v1

Nodes equivalence
(φ function v2 VS v1)

```
eq_node(N1,N2,LI,LI):-
        member(p(N1,N2),LI),!.

eq_node(N1,N2,LI,[p(N1,N2)|LI]):-

        not(member(p(N1,_),LI)),

        not(member(p(_,N2),LI)).
```

```
eq_node(N1,N2):-
        p(N1,N2).
eq_node(N1,_):-
        p(N1,_),!,
        fail.
eq_node(_,N2):-
        p(_,N2),!,
        fail.
eq_node(N1,N2):-
        asserta(p(N1,N2)).
eq_node(N1,N2):-
        retract(p(N1,N2)),!,
        fail.
```

Nodes equivalence
(implements φ function)
V3 – with arguments (incomplete lists)

- Same as v2 just that lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the “not found” case
- Just the equivalence changes (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node(N1,N2,[p(N1,N2)|_]):-!.
eq_node(N1,_,[p(N1,_)|_]):- !,
      fail.
eq_node(_,N2,[p(_,N2)|_]):- !,
      fail.
eq_node(N1,N2,[_|T]):-
      eq_node(N1,N2,T).
```

- Compare v3 with v1. Why is 1 (from v1) clause missing (in v3)?
- Compare v3 with v2.

Nodes equivalence

(φ function v3 VS v1)

```
eq_node(N1,N2,[p(N1,N2)|_]):-!.
eq_node(N1,_,[p(N1,_)|_]):-!,
        fail.
eq_node(_,N2,[p(_,N2)|_]):-!,
        fail.
eq_node(N1,N2,[_|T]):-

        eq_node(N1,N2,T).
```

```
eq_node(N1,N2):-
        p(N1,N2).
eq_node(N1,_):-
        p(N1,_),!,
        fail.
eq_node(_,N2):-
        p(_,N2),!,
        fail.
eq_node(N1,N2):-
        asserta(p(N1,N2)).
eq_node(N1,N2):-
        retract(p(N1,N2)),!,
        fail.
```

Nodes equivalence
(φ function v3 VS v2)

```
eq_node(N1,N2,[p(N1,N2)|_]):-!.
eq_node(N1,_,[p(N1,_)|_]):-!,
        fail.
eq_node(_,N2,[p(_,N2)|_]):-!,
        fail.
eq_node(N1,N2,[_|T]):-

        eq_node(N1,N2,T).
```

```
eq_node(N1,N2,LI,LI):-
        member(p(N1,N2),LI),!.

eq_node(N1,N2,LI,[p(N1,N2)|LI]):-

        not(member(p(N1,_),LI)),

        not(member(p(_,N2),LI)).
```

Nodes equivalence
(implements φ function)
V4 – with arguments (incomplete lists)

- Same as v2 just that lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the "not found" case
- Just the equivalence changes (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node4(N1,N2,LI):-
        member_il(p(N1,N2),LI),!.
eq_node4(N1,N2,[p(N1,N2)|LI]):-
        not(member_il(p(N1,_),LI)),
        not(member_il(p(_,N2),LI)).

member_il(_, L):-var(L), !, fail.
member_il(X, [X|_]):-!.
member_il(X, [_|T]):-member_il(X, T).
```

- Compare v4 with v2.

Nodes equivalence
(φ function v4 VS v2)

```
eq_node(N1,N2,LI):-
        member_il(p(N1,N2),LI),!.

eq_node(N1,N2,[p(N1,N2)|LI]):-

        not(member_il(p(N1,_),LI)),

        not(member_il(p(_,N2),LI)).
```

```
eq_node(N1,N2,LI,LI):-
        member(p(N1,N2),LI),!.

eq_node(N1,N2,LI,[p(N1,N2)|LI]):-

        not(member(p(N1,_),LI)),

        not(member(p(_,N2),LI)).
```

Hamiltonian cycle

- Consider the undirected weighted graph:

```
edge(a,b,7).
edge(a,c,1).
edge(a,d,8).
edge(b,c,2).
edge(b,f,5).
edge(b,e,10).
edge(c,d,3).
edge(d,f,4).
edge(d,e,8).
edge(f,e,3).

is_edge(X,Y,W):-
        edge(X,Y,W);
        edge(Y,X,W).
```

Hamiltonian cycle (NP complete/NPhard problem)

[q] ?- hamilton(6,a,Cycle,Cost).

```
hamilton(N,X,Cycle,Cost):-
      N1 is N-1,
      try(N1,X,X,[X],Cycle,0,Cost).

try(N,X,Y,Pway,Cycle,Pcost,Cost):-
      is_edge(Y,Z,W),
      not(member(Z,Pway)),
      N>0,
      N1 is N-1,
      NewPcost is Pcost+W,
      try(N1,X,Z,[Z|Pway],Cycle,NewPcost,Cost).
try(0,X,Y,Pway,[X|Pway],Pcost,Cost):-
      is_edge(Y,X,W),
      Cost is Pcost+W.
```

Hamiltonian cycle (NP complete/NPhard problem)

- Should not be addressed straight away unless the space is VERY small (how small? Size ~20? TBD orally)
- What should be done? One approach next lecture!

BFS – queues explicit

- BFS with 3 arguments representing:
  - Candidate list (`Cand`) = queue = list of items we reached, yet not finalized to process = grey nodes. Implemented as incomplete lists. Why? Try to find out. Oral discussion!
  - Expanded list (`Exp`) = list of items reached and processed = black nodes. Implemented as regular (complete) lists.
  - Result (`Rez`) = nodes in order of bfs processing. Copy of Expanded list when processing ends.

```
% bf_search/3 (Queue_cand, List_expand, Result).

[q] ?- bf_search([a|_], [], R).

bf_search(Cand,Exp,Exp):-                   % when Q is empty, end exe, the Exp becomes Rez
        var(Cand),!.
bf_search([X|Cand],Exp,R):-                 % else, take first from Q
        expand(X,Cand,Exp),                 % and expand (put all white neighbors in Q) and
        bf_search(Cand,[X|Exp],R).          % continue by moving it in expanded
                                            % = make it black
```

BFS – queues explicit - contd

- The expansion step (2-stage process) – decides which neighbors to add in Q
- dynamic predicate `desc`; potential neighbors stored in auxiliary knowledge base (akb)

```
expand(X,_,Exp):-
        is_edge(X,Z),            % nondeterministically take Z, first neighbor of X
                                 % (eventually all of them)
        not(member(Z,Exp)),      % should NOT be already processed (not a black node)
        assertz(desc(Z)),        % potentially add it in Q
        fail.                    % backtrack to evaluate another neighbor of X
expand(_,Cand,_):-
        assertz(desc(end)),      % mark end of akb
        collect(Cand).

collect(Cand):-
        get_next(X),!,           % take X and if not under processing (not a grey one)
                                 % as long as akb not empty
        insert_IL(X,Cand),       % add in Q
        collect(Cand).           % continue. How is possible with the SAME argument?
collect(Cand).                   % end when akb empty
```

BFS – queues explicit - contd

```
get_next(X):-
        retract(desc(X)),!,
        X\=end.

insert_IL(X,[X|_]):-!.
insert_IL(X,[_|L]):-
        insert_IL(X,L).
```

[q] `?- bf_search([a|_], [], R).` % for the first graph – search for a path

BFS – complete code

```
bf_search(Cand,Exp,Exp):-
        var(Cand),!.
bf_search([X|Cand],Exp,Rez):-
        expand(X,Cand,Exp),
        bf_search(Cand,[X|Exp],Rez).

expand(X,_,Exp):-
        is_edge(X,Z),
        not(member(Z, Exp)),
        assertz(desc(Z)),
        fail.
expand(_,Cand,_):-
        assertz(desc(end)),
        collect(Cand).
```

```
collect(Cand):-
        get_next(X),!,
        insert_IL(X,Cand),
        collect(Cand).
collect(Cand).

get_next(X):-
        retract(desc(X)),!
        X\=end.

insert_IL(X,[X|_]):-!.
insert_IL(X,[_|L]):-
        insert_IL(X,L).
```

[q] `?- bf_search([a|_],[],Rez).`

BFS – execution

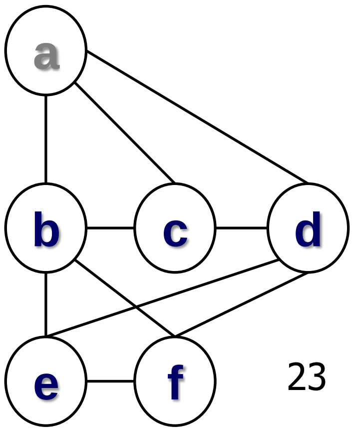
`[q] ?- bf_search([a|_],[],Rez).` % for the first graph – search for a path

Step1

Exp: []

Cand: [a|_]

```
edge(a,b,7).
edge(a,c,1).
edge(a,d,8).
edge(b,c,2).
edge(b,f,5).
edge(b,e,10).
edge(c,d,3).
edge(d,f,4).
edge(d,e,8).
edge(f,e,3).
```

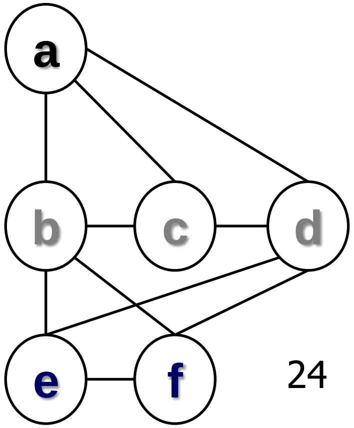


BFS – execution

[q] `?- bf_search([a|_],[],Rez).` % for the first graph – search for a path

	Step1	Step2
Exp:	[]	[a]
Cand:	[a|_]	[b,c,d|_]

```
edge(a,b,7).
edge(a,c,1).
edge(a,d,8).
edge(b,c,2).
edge(b,f,5).
edge(b,e,10).
edge(c,d,3).
edge(d,f,4).
edge(d,e,8).
edge(f,e,3).
```

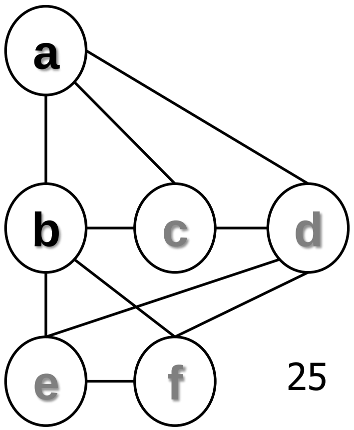


BFS – execution

`[q] ?- bf_search([a|_],[],Rez).` % for the first graph – search for a path

	Step1	Step2	Step3
Exp:	[]	[a]	[b,a]
Cand:	[a|_]	[b,c,d|_]	[c,d,f,e|_]

```
edge(a,b,7).
edge(a,c,1).
edge(a,d,8).
edge(b,c,2).
edge(b,f,5).
edge(b,e,10).
edge(c,d,3).
edge(d,f,4).
edge(d,e,8).
edge(f,e,3).
```

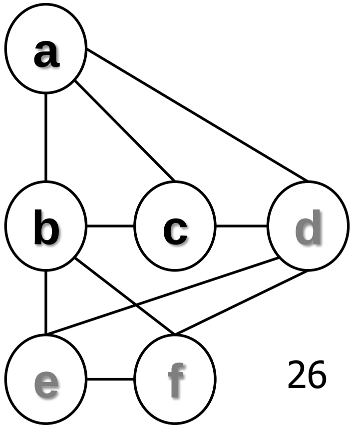


BFS – execution

`[q] ?- bf_search([a|_],[],Rez).` % for the first graph – search for a path

	Step1	Step2	Step3	Step4
Exp:	[]	[a]	[b,a]	[c,b,a]
Cand:	[a|_]	[b,c,d|_]	[c,d,f,e|_]	[d,f,e|_]

```
edge(a,b,7).
edge(a,c,1).
edge(a,d,8).
edge(b,c,2).
edge(b,f,5).
edge(b,e,10).
edge(c,d,3).
edge(d,f,4).
edge(d,e,8).
edge(f,e,3).
```

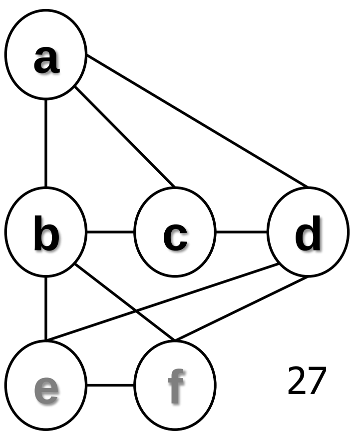


BFS – execution

`[q] ?- bf_search([a|_],[],Rez).` % for the first graph – search for a path

	Step1	Step2	Step3	Step4	Step5
Exp:	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]
Cand:	[a|_]	[b,c,d|_]	[c,d,f,e|_]	[d,f,e|_]	[f,e|_]

```
edge(a,b,7).
edge(a,c,1).
edge(a,d,8).
edge(b,c,2).
edge(b,f,5).
edge(b,e,10).
edge(c,d,3).
edge(d,f,4).
edge(d,e,8).
edge(f,e,3).
```

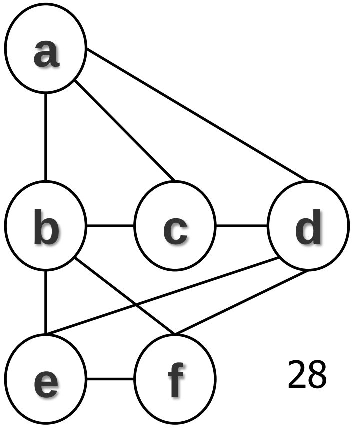


BFS – execution

`[q] ?- bf_search([a|_],[],Rez).` % for the first graph – search for a path

	Step1	Step2	Step3	Step4	Step5	Step6	Step7
Exp :	[]	[a]	[b,a]	[c,b,a]	[d,c,b,a]	[f,d,c,b,a]	[e,f,d,c,b,a] DONE!
Cand:	[a|_]	[b,c,d|_]	[c,d,f,e|_]	[d,f,e|_]	[f,e|_]	[e|_]	_

```
edge(a,b,7).
edge(a,c,1).
edge(a,d,8).
edge(b,c,2).
edge(b,f,5).
edge(b,e,10).
edge(c,d,3).
edge(d,f,4).
edge(d,e,8).
edge(f,e,3).
```



Next time

- B&B solutions to address complexity issues